UNIVERSIDADE CATÓLICA DE BRASÍLIA

Laboratório de Banco de Dados — GPE17M40083

Prof. Samuel Novais Moura Júnior

Projeto Final — Relatório Consolidado

Escola de Música: do modelo conceitual à aplicação

Etapa 1 (projeto e construção) + Etapa 2 (operação, interface e desempenho)

Grupo: Group 8

Integrantes

Gabriel Gomes Giani
Paulo Henrique Barreto da Rocha Dantas
Letícia Paolino
Artur Henrique Caetano
Lucas de Oliveira Faustino

SGBD: MySQL 8 · Aplicação: Python + Flask

Repositório: github.com/gabrielggiani-alt/escola-de-musica-db

Brasília, outubro de 2026

Sumário

Parte I — Etapa 1: projeto e construção do banco **3**
1. Requisitos mínimos de complexidade 4
2. A1 — Escopo e regras de negócio 5
3. A2 — Modelo Entidade-Relacionamento conceitual 8
4. A3 — Dicionário de dados conceitual 9
5. A4 — Modelo lógico relacional 14
6. A5 — Verificação de normalização 17
7. Implementação física, carga e consultas (A6 a A8) 20
8. Declaração de uso de inteligência artificial 22

Parte II — Etapa 2: operação, interface e desempenho **23**
1. Como reproduzir tudo do zero 23
2. B1 — Aplicação com interface 23
3. B2 — Camada de acesso a dados 24
4. B3 — Transações e controle de concorrência 24
5. B4 — Otimização de consultas 26
6. B5 — Segurança e controle de acesso 26
7. B6 — Gatilhos e auditoria 28
8. B7 — Projeto físico e armazenamento 28
9. Alterações no modelo desde a Etapa 1 30
10. Roteiro da demonstração 31
11. Declaração de uso de inteligência artificial 32

Os números deste sumário são a numeração geral do arquivo, carimbada no canto superior direito de cada página. O rodapé de cada parte mantém a numeração do relatório original da etapa.

UNIVERSIDADE CATÓLICA DE BRASÍLIA
Laboratório de Banco de Dados — GPE17M40083
Prof. Samuel Novais Moura Júnior

Projeto Final — Etapa 1

Escola de Música: projeto e construção do banco de dados

Integrantes
Gabriel Gomes Giani — UC21200531
Paulo Henrique Barreto da Rocha Dantas — UC25200677
Letícia Paolino — matrícula não informada
Artur Henrique Caetano — UC25200566
Lucas de Oliveira Faustino — UC25200756

SGBD: MySQL 8.4 · Notação do MER: Engenharia da Informação (pé de galinha)

Brasília, setembro de 2026

Sumário

1. Requisitos mínimos de complexidade
2. A1 — Escopo e regras de negócio
3. A2 — Modelo Entidade-Relacionamento conceitual
4. A3 — Dicionário de dados conceitual
5. A4 — Modelo lógico relacional
6. A5 — Verificação de normalização
7. Implementação física, carga e consultas (A6 a A8)
8. Declaração de uso de inteligência artificial

1. Requisitos mínimos de complexidade

Requisito do enunciado	Como o modelo atende
Entidades (mínimo 8, sem associativas)	11: pessoa, aluno, professor, instrumento, nivel, sala, turma, aula, avaliacao, mensalidade, historico_situacao
Relacionamentos N:N com atributo (mínimo 2)	3: habilitacao, matricula, frequencia
Autorrelacionamento (mínimo 1)	nivel → nivel (pré-requisito)
Generalização/especialização (mínimo 1)	pessoa → aluno / professor, total e sobreposta, 3 tabelas (ver A4)
Entidade fraca (mínimo 1)	avaliacao, identificada por (id_matricula, numero_avaliacao)
Atributo temporal (mínimo 1)	historico_situacao (situação da matrícula ao longo do tempo); frequência aula a aula
Regras de negócio (mínimo 20)	30 regras, RN01 a RN30
Volume de carga (40 nas principais, 100 na maior)	pessoa 54, aluno 47, matricula 71, historico 110, avaliacao 127, aula 167, mensalidade 246, frequencia 901

2. A1 — Escopo e regras de negócio

Descrição do domínio

O sistema atende uma escola de música de Brasília que oferece cursos de violão, piano, violino, bateria, canto e flauta transversal. Cada curso é dividido em **níveis** sequenciais (Básico I, Básico II, Intermediário). Um nível pode exigir que o aluno tenha concluído o nível anterior, e é esse encadeamento que organiza a trajetória do aluno na escola.

A escola cadastra **pessoas** com dados de identificação (CPF, nascimento), contato (e-mail e vários telefones) e endereço. Toda pessoa cadastrada é **aluno**, **professor** ou as duas coisas: é comum um professor da casa estudar outro instrumento. Do aluno interessa a data de ingresso e, se for menor de idade, o responsável. Do professor interessam a admissão, o valor da hora-aula, a formação e os instrumentos em que é **habilitado**, com o grau de proficiência e a data da certificação.

A cada semestre a escola abre **turmas**. Cada turma oferta um nível, tem um professor habilitado naquele instrumento, uma **sala** (com capacidade e, em algumas, piano), um dia da semana, um horário e um número de vagas. Os alunos se inscrevem por **matrícula**, que tem data e uma situação: ativa, concluída, trancada ou cancelada. Como a situação muda com o tempo (um aluno tranca por viagem e depois volta), cada mudança é guardada com data, hora e motivo no **histórico de situação**.

Cada encontro da turma é uma **aula**, numerada e datada, que pode ser realizada ou cancelada. Em cada aula realizada o professor registra a **frequência** de cada aluno, com justificativa quando há falta. Ao longo do nível o aluno faz **avaliações** teóricas, práticas e um recital, que só existem dentro da matrícula. É aprovado quem termina com média de pelo menos 6,0 e frequência de pelo menos 75%.

No financeiro, cada matrícula gera uma **mensalidade** por mês de competência, com valor e vencimento. Quando paga, registram-se a data e a forma de pagamento; enquanto não paga, fica em aberto, o que permite acompanhar a inadimplência.

Fora do escopo desta etapa: folha de pagamento dos professores, venda de instrumentos, controle de acesso e login (Etapa 2).

Regras de negócio

“Atendida por” indica se a regra é garantida por **restrição** do banco (o SGBD rejeita o dado inválido), por **consulta** (a consulta indicada lista as violações e deve voltar vazia) ou pela **aplicação** (a verificação depende de algo que o CHECK do MySQL não aceita, como a data atual ou dados de outra tabela).

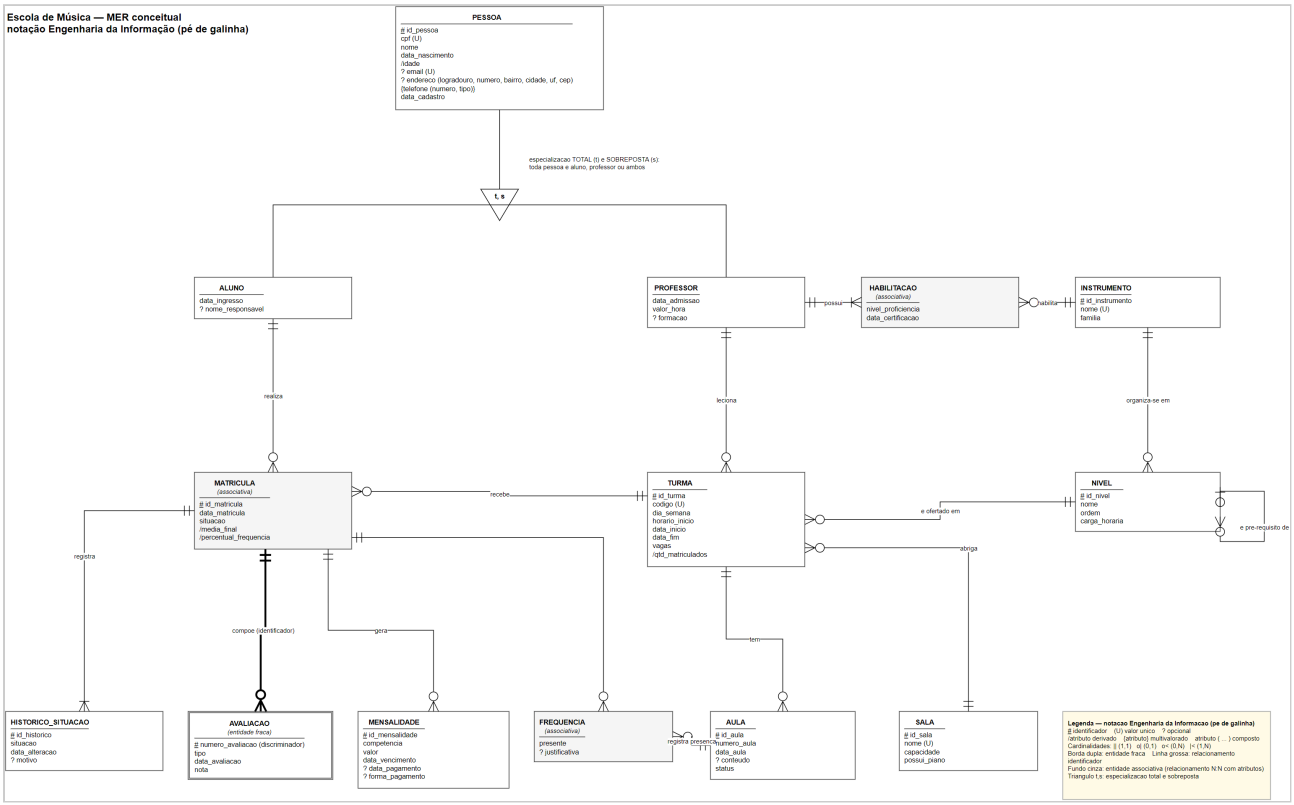
Código	Regra	Atendida por	Onde
RN01	O CPF de uma pessoa é único no cadastro e tem exatamente 11 dígitos numéricos.	Restrição	`uq_pessoa_cpf, ck_pessoa_cpf`
RN02	O e-mail é opcional, mas quando informado não pode se repetir entre pessoas.	Restrição	`uq_pessoa_email`

Código	Regra	Atendida por	Onde
RN03	A data de nascimento não é anterior a 01/01/1920 nem posterior à data do cadastro.	Restrição + aplicação	`ck_pessoa_nascimento (limite inferior); o limite “não futura” fica na aplicação`
RN04	Todo aluno menor de 18 anos tem um responsável registrado.	Consulta	`C15 (auditoria) e C01`
RN05	O valor da hora-aula de um professor é maior que zero.	Restrição	`ck_professor_valor_hora`
RN06	O nome de um instrumento é único, e sua família é CORDAS, SOPRO, PERCUSSAO, TECLAS ou VOZ.	Restrição	`uq_instrumento_nome, ck_instrumento_familia`
RN07	A proficiência de um professor em um instrumento é INTERMEDIARIO, AVANCADO ou ESPECIALISTA.	Restrição	`ck_habilitacao_proficiencia`
RN08	Dentro de um instrumento, cada posição (1 a 10) na sequência de níveis é ocupada por um único nível.	Restrição	`uq_nivel_instrumento_ordem, ck_nivel_ordem`
RN09	Um nível pode exigir no máximo um nível como pré-requisito; um nível que é pré-requisito de outro não pode ser excluído.	Restrição	`fk_nivel_prerequisito (ON DELETE RESTRICT)`
RN10	A capacidade de uma sala fica entre 1 e 30 pessoas.	Restrição	`ck_sala_capacidade`
RN11	A escola funciona de segunda a sábado, e as aulas começam entre 08h e 21h.	Restrição	`ck_turma_dia, ck_turma_horario`
RN12	A data de término de uma turma é posterior à data de início.	Restrição	`ck_turma_periodo`
RN13	Uma turma oferece de 1 a 20 vagas, sem ultrapassar a capacidade da sala onde acontece.	Restrição + aplicação	`ck_turma_vagas; a comparação com a sala fica na aplicação`
RN14	Um aluno não pode ter duas matrículas na mesma turma.	Restrição	`uq_matricula_aluno_turma`
RN15	A situação de uma matrícula é ATIVA, CONCLUIDA, TRANCADA ou CANCELADA.	Restrição	`ck_matricula_situacao, ck_historico_situacao`
RN16	Um aluno que possui matrícula não pode ser excluído do cadastro.	Restrição	`fk_matricula_aluno (ON DELETE RESTRICT)`
RN17	Toda mudança de situação de uma matrícula fica registrada com data e hora, e duas mudanças da mesma matrícula não ocorrem no mesmo instante.	Restrição + aplicação	`tabela historico_situacao, uq_historico_matricula_data; o registro é feito pela aplicação (na Etapa 2, por gatilho)`

Código	Regra	Atendida por	Onde
RN18	Dentro de uma turma, o número e a data de cada aula não se repetem.	Restrição	`uq_aula_turma_numero, uq_aula_turma_data`
RN19	A presença de um aluno é registrada uma única vez por aula.	Restrição	`pk_frequencia`
RN20	Só existe justificativa para falta; presença não tem justificativa.	Restrição	`ck_frequencia_justificativa`
RN21	Uma avaliação é do tipo PRATICA, TEORICA ou RECITAL.	Restrição	`ck_avaliacao_tipo`
RN22	A nota de uma avaliação vai de 0 a 10.	Restrição	`ck_avaliacao_nota`
RN23	Existe no máximo uma mensalidade por matrícula em cada mês de competência, e a competência é sempre o dia 1 do mês.	Restrição	`uq_mensalidade_matricula_competencia, ck_mensalidade_competencia`
RN24	O valor da mensalidade é positivo, e o vencimento cai no mês de competência ou depois.	Restrição	`ck_mensalidade_valor, ck_mensalidade_vencimento`
RN25	Mensalidade paga tem data e forma de pagamento (PIX, CARTAO, BOLETO ou DINHEIRO); mensalidade em aberto não tem nenhuma das duas.	Restrição	`ck_mensalidade_pagamento`
RN26	Toda pessoa cadastrada é aluno, professor ou ambos.	Consulta	`C15 (auditoria)`
RN27	O professor de uma turma é habilitado no instrumento do nível daquela turma.	Consulta	`C15 (auditoria)`
RN28	Um aluno só se matricula em um nível com pré-requisito se tiver matrícula CONCLUIDA no nível pré-requisito.	Consulta	`C15 (auditoria)`
RN29	O aluno é aprovado no nível quando tem média das avaliações maior ou igual a 6,0 e frequência maior ou igual a 75%.	Consulta	`C11`
RN30	A frequência de uma matrícula só é lançada em aulas da turma daquela matrícula.	Consulta + aplicação	`C15 (auditoria); a tela de chamada lista apenas os alunos da turma`

3. A2 — Modelo Entidade-Relacionamento conceitual

Notação de Engenharia da Informação (pé de galinha), mantida em todo o diagrama. O arquivo-fonte editável é `docs/mer-conceitual.drawio` (abre no draw.io), e a exportação em PDF está em `docs/mer-conceitual.pdf`.



Leitura do diagrama

Especialização: PESSOA → ALUNO / PROFESSOR, **total** (não existe pessoa sem papel) e **sobreposta** (a mesma pessoa pode ser as duas). **Autorrelacionamento:** NIVEL “é pré-requisito de” NIVEL, (0,1) para (0,N). **Entidade fraca:** AVALIACAO, de borda dupla, ligada a MATRICULA pelo relacionamento identificador de linha grossa; seu discriminador é numero_avaliacao. **Associativas** (N:N com atributos, fundo cinza): HABILITACAO, MATRICULA e FREQUENCIA. **Atributo composto:** endereco; **multivalorado:** {telefone}; **derivados:** /idade, /media_final, /percentual_frequencia, /qtd_matriculados.

Cardinalidades principais: um aluno realiza (0,N) matrículas, e cada matrícula é de exatamente (1,1) aluno. Uma turma recebe (0,N) matrículas. Um professor possui (1,N) habilitações, porque só é professor quem é habilitado em algo. Uma matrícula registra (1,N) mudanças de situação, porque ela nasce com o registro “ATIVA”. Uma turma tem (0,N) aulas, e cada aula registra (0,N) presenças.

4. A3 — Dicionário de dados conceitual

Formato do Anexo A do enunciado. A coluna Observação traz a regra de negócio associada. Atributos marcados com "/" são derivados e não são armazenados.

PESSOA

Qualquer pessoa cadastrada na escola. Superclasse da especialização total e sobreposta em ALUNO e PROFESSOR.

Atributo	Descrição	Domínio	Obrig.	Observação
`id_pessoa`	Identificador interno da pessoa	Inteiro sequencial	Sim	Chave primária substituta
`cpf`	Cadastro de pessoa física	11 dígitos numéricos	Sim	RN01 — não pode repetir
`nome`	Nome completo	Texto de até 100 caracteres	Sim	
`data_nascimento`	Data de nascimento	Data a partir de 01/01/1920	Sim	RN03
`/idade`	Idade em anos completos	Inteiro, calculado	—	Derivado de data_nascimento; não armazenado
`email`	E-mail de contato	Texto de até 120 caracteres	Não	RN02 — único quando informado
`endereco`	Endereço residencial (composto)	logradouro (120), numero (10), bairro (60), cidade (60), uf (2 letras), cep (8 dígitos)	Não	Atributo composto; aceita ausência
`{telefone}`	Telefones de contato (multivalorado)	número com DDD (10 ou 11 dígitos) + tipo CELULAR, FIXO ou RECADO	Não	Uma pessoa pode ter vários
`data_cadastro`	Data em que a pessoa entrou no cadastro	Data	Sim	Padrão: data atual

ALUNO

Pessoa que estuda na escola (subclasse de PESSOA).

Atributo	Descrição	Domínio	Obrig.	Observação
`data_ingresso`	Data da primeira matrícula ou inscrição	Data	Sim	
`nome_responsavel`	Responsável legal pelo aluno	Texto de até 100 caracteres	Não	RN04 — obrigatório para menor de 18 anos

PROFESSOR

Pessoa que leciona na escola (subclasse de PESSOA).

Atributo	Descrição	Domínio	Obrig.	Observação
`data_admissao`	Data de admissão na escola	Data	Sim	

Atributo	Descrição	Domínio	Obrig.	Observação
valor_hora	Valor pago por hora-aula	Decimal em reais, até 99.999,99	Sim	RN05 — maior que zero
formacao	Formação acadêmica ou técnica	Texto de até 100 caracteres	Não	

INSTRUMENTO

Instrumento musical (ou voz) ensinado pela escola.

Atributo	Descrição	Domínio	Obrig.	Observação
id_instrumento	Identificador interno do instrumento	Inteiro sequencial	Sim	Chave primária substituta
nome	Nome do instrumento	Texto de até 40 caracteres	Sim	RN06 — único
familia	Família do instrumento	CORDAS, SOPRO, PERCUSSAO, TECLAS ou VOZ	Sim	RN06

HABILITACAO

Associativa: habilitação de um professor em um instrumento (N:N PROFESSOR × INSTRUMENTO).

Atributo	Descrição	Domínio	Obrig.	Observação
nivel_proficiencia	Grau de domínio do professor no instrumento	INTERMEDIARIO, AVANCADO ou ESPECIALISTA	Sim	RN07
data_certificacao	Data em que obteve a certificação	Data	Sim	

NIVEL

Etapa do curso de um instrumento. Possui autorrelacionamento de pré-requisito.

Atributo	Descrição	Domínio	Obrig.	Observação
id_nivel	Identificador interno do nível	Inteiro sequencial	Sim	Chave primária substituta
nome	Nome do nível	Texto de até 40 caracteres	Sim	Ex.: Violão Básico II
ordem	Posição do nível na sequência do instrumento	Inteiro de 1 a 10	Sim	RN08 — única por instrumento
carga_horaria	Carga horária total do nível	Inteiro de 8 a 200 horas	Sim	
(pré-requisito)	Nível que precisa ser concluído antes	Referência a outro NIVEL	Não	RN09, RN28 — autorrelacionamento (0,1)

SALA

Espaço físico onde as turmas acontecem.

Atributo	Descrição	Domínio	Obrig.	Observação
id_sala	Identificador interno da sala	Inteiro sequencial	Sim	Chave primária substituta

Atributo	Descrição	Domínio	Obrig.	Observação
nome	Nome da sala	Texto de até 30 caracteres	Sim	Único
capacidade	Número máximo de pessoas	Inteiro de 1 a 30	Sim	RN10
possui_piano	Indica se a sala tem piano	Verdadeiro ou falso	Sim	Padrão: falso

TURMA

Oferta de um nível em um semestre, com professor, sala e horário fixos.

Atributo	Descrição	Domínio	Obrig.	Observação
id_turma	Identificador interno da turma	Inteiro sequencial	Sim	Chave primária substituta
codigo	Código legível da turma	Texto de até 12 caracteres, ex.: VIO-B2-26B	Sim	Único
dia_semana	Dia da semana da aula	Inteiro de 2 (segunda) a 7 (sábado)	Sim	RN11
horario_inicio	Horário de início da aula	Hora entre 08:00 e 21:00	Sim	RN11
data_inicio	Primeiro dia de aula	Data	Sim	
data_fim	Último dia do período letivo	Data posterior a data_inicio	Sim	RN12
vagas	Número de vagas oferecidas	Inteiro de 1 a 20	Sim	RN13
/qtd_matriculados	Alunos ativos na turma	Inteiro, calculado	—	Derivado; calculado na consulta C08

MATRICULA

Associativa: vínculo de um aluno com uma turma (N:N ALUNO × TURMA). Dona das avaliações, frequências e mensalidades.

Atributo	Descrição	Domínio	Obrig.	Observação
id_matricula	Identificador interno da matrícula	Inteiro sequencial	Sim	Chave primária substituta
data_matricula	Data em que a matrícula foi feita	Data	Sim	
situacao	Situação atual da matrícula	ATIVA, CONCLUIDA, TRANCADA ou CANCELADA	Sim	RN15 — padrão ATIVA
/media_final	Média das avaliações	Decimal de 0 a 10, calculado	—	Derivado; consulta C11 (RN29)
/percentual_frequencia	Percentual de presença nas aulas realizadas	0 a 100%, calculado	—	Derivado; consulta C11 (RN29)

HISTORICO_SITUACAO

Registro datado de cada situação por que a matrícula passou (atributo temporal).

Atributo	Descrição	Domínio	Obrig.	Observação
`id_historico`	Identificador interno do registro	Inteiro sequencial	Sim	Chave primária substituta
`situacao`	Situação assumida naquele momento	ATIVA, CONCLUIDA, TRANCADA ou CANCELADA	Sim	RN15
`data_alteracao`	Data e hora da mudança	Data e hora	Sim	RN17 — única por matrícula
`motivo`	Motivo da mudança	Texto de até 200 caracteres	Não	

AULA

Encontro de uma turma em uma data.

Atributo	Descrição	Domínio	Obrig.	Observação
`id_aula`	Identificador interno da aula	Inteiro sequencial	Sim	Chave primária substituta
`numero_aula`	Número sequencial da aula na turma	Inteiro de 1 a 255	Sim	RN18 — único na turma
`data_aula`	Data em que a aula ocorreu	Data	Sim	RN18 — única na turma
`conteudo`	Conteúdo trabalhado	Texto de até 200 caracteres	Não	
`status`	Se a aula aconteceu	REALIZADA ou CANCELADA	Sim	Padrão: REALIZADA

FREQUENCIA

Associativa: presença de uma matrícula em uma aula (N:N MATRICULA × AULA). Tabela de maior movimento.

Atributo	Descrição	Domínio	Obrig.	Observação
`presente`	Se o aluno esteve na aula	Verdadeiro ou falso	Sim	RN19
`justificativa`	Justificativa da falta	Texto de até 200 caracteres	Não	RN20 — só em falta

AVALIACAO

Entidade fraca: avaliação de desempenho, que só existe dentro de uma matrícula.

Atributo	Descrição	Domínio	Obrig.	Observação
`numero_avaliacao`	Número da avaliação dentro da matrícula (discriminador)	Inteiro de 1 a 10	Sim	Identifica junto com a matrícula
`tipo`	Tipo de avaliação	PRATICA, TEORICA ou RECITAL	Sim	RN21
`data_avaliacao`	Data da avaliação	Data	Sim	
`nota`	Nota obtida	Decimal de 0,00 a 10,00	Sim	RN22

MENSALIDADE

Cobrança mensal referente a uma matrícula.

Atributo	Descrição	Domínio	Obrig.	Observação
`id_mensalidade`	Identificador interno da mensalidade	Inteiro sequencial	Sim	Chave primária substituta
`competencia`	Mês de referência	Data, sempre dia 1	Sim	RN23 — única por matrícula
`valor`	Valor cobrado	Decimal em reais, maior que zero	Sim	RN24
`data_vencimento`	Data de vencimento	Data igual ou posterior à competência	Sim	RN24
`data_pagamento`	Data em que foi paga	Data	Não	RN25 — ausente = em aberto
`forma_pagamento`	Meio de pagamento	PIX, CARTAO, BOLETO ou DINHEIRO	Não	RN25 — obrigatória se paga

5. A4 — Modelo lógico relacional

Notação: **sublinhado** = chave primária; *itálico* seguido de → = chave estrangeira e tabela referenciada.

PESSOA (**id_pessoa**, nome, cpf, data_nascimento, email, logradouro, numero, bairro, cidade, uf, cep, data_cadastro)

TELEFONE (***id_pessoa***→**pessoa**, **numero**, tipo)

ALUNO (***id_pessoa***→**pessoa**, data_ingresso, nome_responsavel)

PROFESSOR (***id_pessoa***→**pessoa**, data_admissao, valor_hora, formacao)

INSTRUMENTO (**id_instrumento**, nome, familia)

HABILITACAO (***id_professor***→**professor**, ***id_instrumento***→**instrumento**, nivel_proficiencia, data_certificacao)

NIVEL (**id_nivel**, *id_instrumento*→instrumento, nome, ordem, carga_horaria, *id_nivel_prerequisito*→nivel)

SALA (**id_sala**, nome, capacidade, possui_piano)

TURMA (**id_turma**, codigo, *id_nivel*→nivel, *id_professor*→professor, *id_sala*→sala, dia_semana, horario_inicio, data_inicio, data_fim, vagas)

MATRICULA (**id_matricula**, *id_aluno*→aluno, *id_turma*→turma, data_matricula, situacao)

HISTORICO_SITUACAO (**id_historico**, *id_matricula*→matricula, situacao, data_alteracao, motivo)

AULA (**id_aula**, *id_turma*→turma, numero_aula, data_aula, conteudo, status)

FREQUENCIA (***id_matricula***→**matricula**, ***id_aula***→**aula**, presente, justificativa)

AVALIACAO (***id_matricula***→**matricula**, **numero_avaliacao**, tipo, data_avaliacao, nota)

MENSALIDADE (**id_mensalidade**, *id_matricula*→matricula, competencia, valor, data_vencimento, data_pagamento, forma_pagamento)

Onde está cada N:N

Relacionamento N:N no MER	Tabela associativa	Chaves estrangeiras	Atributos próprios
PROFESSOR × INSTRUMENTO	habilitacao	id_professor → professor, id_instrumento → instrumento	nivel_proficiencia, data_certificacao
ALUNO × TURMA	matricula	id_aluno → aluno, id_turma → turma	data_matricula, situacao
MATRICULA × AULA	frequencia	id_matricula → matricula, id_aula → aula	presente, justificativa

Decisões de mapeamento

Especialização PESSOA → ALUNO / PROFESSOR (total, sobreposta). Mapeada em **três tabelas**: uma para a superclasse (pessoa) e uma para cada subclasse (aluno, professor), com a chave da superclasse repetida como PK e FK. Por que não **tabela única**: a especialização é sobreposta (a pessoa 6 é professora e também aluna de piano); uma tabela só teria colunas de aluno e de professor quase sempre nulas e não permitiria que matrícula apontasse “só para alunos” nem que turma apontasse “só para professores”. Por que não **uma tabela por subclasse sem superclasse**: quem é as duas coisas teria nome, CPF e endereço gravados duas vezes, e o CPF único deixaria de ser garantido pelo banco. Com três tabelas, fk_matricula_aluno e fk_turma_professor garantem que só

aluno se matricula e só professor leciona. Limitação assumida: a totalidade (toda pessoa ter pelo menos um papel) não é garantida pelo DDL; é verificada pela consulta C15 (RN26).

Relacionamentos 1:1. Os únicos 1:1 do modelo são pessoa–aluno e pessoa–professor, que vêm da especialização. Eles foram **mantidos separados** pelos motivos acima. Não há outro 1:1 no domínio: um professor tem várias turmas, uma matrícula tem várias mensalidades, e assim por diante.

Relacionamentos N:N. Os três N:N viraram tabelas associativas, cada uma com as FKs para os dois lados e os atributos próprios da associação: **habilitacao** (professor × instrumento, com proficiência e data), **matricula** (aluno × turma, com data e situação) e **frequencia** (matrícula × aula, com presença e justificativa).

Autorrelacionamento. O pré-requisito entre níveis (0,1 para 0,N) virou a coluna **id_nivel_prerequisito** em nivel, anulável e com FK para a própria tabela. Não foi preciso tabela à parte, porque cada nível tem no máximo um pré-requisito.

Entidade fraca. AVALIACAO recebeu a PK **(id_matricula, numero_avaliacao)**, que inclui a chave da entidade dona, com ON DELETE CASCADE: se a matrícula deixa de existir, suas avaliações também deixam.

Atributos compostos, multivalorados e derivados. O endereço composto foi **achatado** em colunas (logradouro, número, bairro, cidade, UF, CEP), porque cada parte é consultada isoladamente (a consulta C02 filtra por bairro). O telefone multivalorado virou a tabela **telefone**. Os derivados (idade, média, percentual de frequência, quantidade de matriculados) **não são armazenados**: são calculados nas consultas, para nunca ficarem desatualizados.

Atributo temporal. A situação da matrícula ao longo do tempo fica em **historico_situacao**, uma linha por mudança com data e hora. A coluna matricula.situacao guarda só a situação atual, para leitura rápida; ver a seção A5 sobre essa redundância.

Ações referenciais (ON DELETE / ON UPDATE). ON UPDATE RESTRICT em todas as FKs: as chaves são substitutas e nunca mudam. **ON DELETE RESTRICT** como padrão, para preservar o histórico acadêmico e financeiro (RN16): não se apaga aluno com matrícula, nem turma com aulas, nem nível que é pré-requisito. **ON DELETE CASCADE** só onde o filho não existe sem o pai: aluno e professor (subclasses), telefone (atributo multivalorado) e avaliação (entidade fraca).

Chave natural × chave substituta

Relação	Escolha	Justificativa
`pessoa`	Substituta (id_pessoa); CPF fica como chave alternativa (UNIQUE)	O CPF é natural, mas é dado pessoal: espalhá-lo como FK em aluno, professor, telefone e matrícula multiplicaria a exposição. Uma correção de CPF digitado errado também exigiria atualizar todas as tabelas filhas.
`telefone`	Natural composta (id_pessoa, numero)	Ninguém referencia telefone; o próprio número dentro da pessoa já identifica a linha. Criar id seria coluna sem uso.
`aluno / professor`	Herdada da superclasse (id_pessoa)	Na especialização, a subclasse usa a mesma chave da superclasse, que é ao mesmo tempo PK e FK. Garante o 1:1 com pessoa.

Relação	Escolha	Justificativa
`instrumento, sala`	Substituta; nome como chave alternativa	O nome poderia ser PK, mas pode ser renomeado (ex.: “Sala 3” vira “Sala Tom Jobim”), e PK não deve mudar.
`habilitacao`	Natural composta (id_professor, id_instrumento)	A combinação é exatamente o que a tabela registra e não pode se repetir; ninguém a referencia.
`nivel`	Substituta; (id_instrumento, ordem) como chave alternativa	É referenciado por turma e pelo próprio autorrelacionamento; uma chave simples deixa essas FKs curtas.
`turma`	Substituta; codigo como chave alternativa	O código é legível para a secretaria, mas segue convenção que pode mudar.
`matricula`	Substituta (id_matricula); (id_aluno, id_turma) como chave alternativa	É referenciada por quatro tabelas (histórico, frequência, avaliação, mensalidade). Carregar a chave composta em todas elas deixaria cada filha maior e as junções mais verbosas.
`historico_situacao, mensalidade`	Substituta; (id_matricula, data_alteracao) e (id_matricula, competencia) como chaves alternativas	Mantém uma chave simples e estável para a aplicação da Etapa 2 atualizar e auditar a linha.
`aula`	Substituta; (id_turma, numero_aula) e (id_turma, data_aula) como chaves alternativas	É referenciada pela frequência, a tabela de maior volume; uma FK de uma coluna só economiza espaço em cada uma das centenas de linhas.
`frequencia`	Natural composta (id_matricula, id_aula)	É a tabela associativa pura: o par é o fato registrado e impede presença duplicada (RN19).
`avaliacao`	Composta com a chave da dona (id_matricula, numero_avaliacao)	Entidade fraca: a identificação depende da matrícula; numero_avaliacao sozinho não identifica nada.

6. A5 — Verificação de normalização

Para cada relação estão listadas as dependências funcionais (DF) relevantes. Critério usado: **1FN** — todos os atributos são atômicos (o endereço composto foi decomposto e o telefone multivalorado virou tabela); **2FN** — nenhum atributo não-chave depende de *parte* de uma chave composta; **3FN** — nenhum atributo não-chave depende de outro atributo não-chave; **FNBC** — todo determinante é chave candidata.

Relação	Dependências funcionais	Forma normal	Análise
`pessoa`	id_pessoa → nome, cpf, data_nascimento, email, logradouro, numero, bairro, cidade, uf, cep, data_cadastro cpf → id_pessoa (chave candidata) email → id_pessoa (chave candidata, quando não nula) cep → logradouro, bairro, cidade, uf (dependência do mundo real, ver abaixo)	**3FN com desnormalização deliberada**	Ver a justificativa sobre o CEP logo abaixo da tabela.
`telefone`	(id_pessoa, numero) → tipo	**FNBC**	Único determinante é a chave.
`aluno`	id_pessoa → data_ingresso, nome_responsavel	**FNBC**	Único determinante é a chave.
`professor`	id_pessoa → data_admissao, valor_hora, formacao	**FNBC**	Único determinante é a chave.
`instrumento`	id_instrumento → nome, familia nome → id_instrumento	**FNBC**	Os dois determinantes são chaves candidatas.
`habilitacao`	(id_professor, id_instrumento) → nivel_proficiencia, data_certificacao	**FNBC**	A proficiência depende do par, não de um lado só; por isso não viola a 2FN.
`nivel`	id_nivel → id_instrumento, nome, ordem, carga_horaria, id_nivel_prerequisito (id_instrumento, ordem) → id_nivel	**FNBC**	Os dois determinantes são chaves candidatas.
`sala`	id_sala → nome, capacidade, possui_piano nome → id_sala	**FNBC**	Os dois determinantes são chaves candidatas.
`turma`	id_turma → codigo, id_nivel, id_professor, id_sala, dia_semana, horario_inicio, data_inicio, data_fim, vagas codigo → id_turma	**FNBC**	O instrumento da turma não foi guardado aqui: ele é obtido por turma → nivel → instrumento. Guardá-lo criaria a dependência transitiva id_turma → id_nivel → id_instrumento.

Relação	Dependências funcionais	Forma normal	Análise
`matricula`	id_matricula → id_aluno, id_turma, data_matricula, situacao (id_aluno, id_turma) → id_matricula	**FNBC**	Os dois determinantes são chaves candidatas.
`historico_situacao`	id_historico → id_matricula, situacao, data_alteracao, motivo (id_matricula, data_alteracao) → id_historico	**FNBC**	Os dois determinantes são chaves candidatas.
`aula`	id_aula → id_turma, numero_aula, data_aula, conteudo, status (id_turma, numero_aula) → id_aula (id_turma, data_aula) → id_aula	**FNBC**	Os três determinantes são chaves candidatas.
`frequencia`	(id_matricula, id_aula) → presente, justificativa	**FNBC**	Presença depende do par inteiro (aluno naquela aula).
`avaliacao`	(id_matricula, numero_avaliacao) → tipo, data_avaliacao, nota	**FNBC**	Entidade fraca; a chave é a da dona mais o discriminador.
`mensalidade`	id_mensalidade → id_matricula, competencia, valor, data_vencimento, data_pagamento, forma_pagamento (id_matricula, competencia) → id_mensalidade	**FNBC**	O valor não é função do nível: é o preço combinado naquele mês, que pode ter desconto ou reajuste. Por isso não há dependência nivel → valor, e o valor fica na própria mensalidade.

Verificação das chaves compostas (2FN)

As relações com chave composta são telefone, habilitacao, frequencia e avaliacao. Em todas elas o atributo não-chave depende da chave *inteira*: o tipo do telefone depende de qual número de qual pessoa; a proficiência depende de qual professor em qual instrumento; a presença depende de qual aluno em qual aula; a nota depende de qual avaliação de qual matrícula. Nenhuma dependência parcial.

Relações deliberadamente desnormalizadas

1. pessoa — CEP. No mundo real o CEP determina logradouro, bairro, cidade e UF, e isso cria uma dependência transitiva id_pessoa → cep → cidade. Normalizar exigiria uma tabela de logradouros dos Correios, com centenas de milhares de linhas, que a escola não tem e não mantém. Além disso o endereço é opcional e digitado pelo próprio aluno, e ler o endereço de uma pessoa passaria a exigir junção. **Decisão:** manter o endereço inteiro em pessoa. O custo aceito é a possibilidade de o mesmo CEP aparecer com cidades diferentes por erro de digitação. A Etapa 2 pode revisitar essa escolha se a aplicação passar a buscar o endereço pelo CEP.

2. matricula.situacao. A situação atual também pode ser obtida do registro mais recente de historico_situacao, então é uma informação redundante (não viola a 3FN, já que depende só da chave, mas é duplicação controlada). Justificativa de desempenho: a situação atual é filtrada em quase toda consulta (C06, C08, C13), e descobrir "o último evento de cada matrícula" exige subconsulta a cada leitura. A consistência entre as duas é garantida pela aplicação e, na Etapa 2, por gatilho.

Relações em FNBC

Todas as relações exceto `pessoa` estão na **Forma Normal de Boyce-Codd**: em cada uma, todo determinante listado é chave candidata.

7. Implementação física, carga e consultas (A6 a A8)

A6 — 01_ddl.sql

Cria o banco `escola_musica` (utf8mb4) e as 15 tabelas, na ordem de dependência. Todas as restrições são nomeadas com os prefixos `pk_ uq_ fk_ ck_ idx_`, e cada restrição que implementa uma regra traz o código dela em comentário. Toda FK declara ON DELETE e ON UPDATE explicitamente.

Observação sobre o MySQL: o MySQL aceita a sintaxe `CONSTRAINT pk_tabela PRIMARY KEY`, mas sempre grava a chave primária com o nome interno `PRIMARY`. O nome pk_ fica no script como documentação. As demais restrições (uq_, fk_, ck_, idx_) ficam gravadas com o nome dado.

Critério dos tipos de dados:

Tipo	Onde	Por quê
INT UNSIGNED	ids de pessoa, turma, matrícula, aula, mensalidade	Tabelas que crescem todo semestre; UNSIGNED dobra a faixa positiva, e id nunca é negativo.
SMALLINT / TINYINT UNSIGNED	instrumento, nível, sala, ordem, vagas, número da aula	Cadastros pequenos e estáveis: 1 ou 2 bytes em vez de 4.
CHAR(11), CHAR(8), CHAR(2)	cpf, cep, uf	Tamanho sempre fixo. São guardados como texto, e não número, para não perder zeros à esquerda.
VARCHAR(n)	nomes, e-mail, códigos, domínios textuais	Tamanho variável, com limite pensado no dado real (e-mail 120, nome 100).
DECIMAL(7,2) / (8,2) / (4,2)	valor_hora, valor da mensalidade, nota	Valor exato. FLOAT gera erro de arredondamento em dinheiro.
DATE / DATETIME / TIME	datas, histórico, horário da turma	O histórico precisa da hora, porque duas mudanças podem ocorrer no mesmo dia; as demais datas só precisam do dia.
BOOLEAN	presente, possui_piano	Sim/não; no MySQL é um TINYINT(1).
VARCHAR + CHECK IN (...)	situação, tipo, família, forma de pagamento	Preferido ao ENUM porque o nome da restrição (ck_) aparece na mensagem de erro, e acrescentar um valor novo não reescreve a tabela.

A7 — 02_carga.sql

Gerado pelo script `ferramentas/gerar_carga.py`, com semente fixa: rodar de novo produz exatamente o mesmo arquivo. Dados fictícios: nomes combinados aleatoriamente, CPFs sem dígito verificador válido e e-mails no domínio reservado `exemplo.com`. A carga cobre os semestres 2026/1 (concluído) e 2026/2 (em andamento até 18/09/2026).

Caso de contorno exigido	Onde aparece na carga
Atributos opcionais em branco	5 pessoas sem e-mail, pessoas sem endereço, 3 pessoas sem telefone, professor sem formação, aulas sem conteúdo, faltas sem justificativa
Situações em aberto	mensalidades em aberto (11 vencidas e não pagas); matrícula TRANCADA em 2026/2; turma VIO-IN-26B sem alunos; nível Flauta Básico I sem turma; professor 8 sem turma

Caso de contorno exigido	Onde aparece na carga
Histórico com mais de um evento	matrícula que foi ATIVA → TRANCADA → ATIVA → CONCLUIDA; todas as matrículas concluídas têm pelo menos 2 eventos; aluno que concluiu 2026/1 e está ativo em 2026/2
Especialização sobreposta	pessoa 6 é professora de violão e aluna de piano
Aula cancelada	3 aulas CANCELADA, sem frequência lançada

A8 — 03_consultas.sql

#	Categoria	Pergunta de negócio	Recurso exigido
C01	Básica	Quais alunos são menores e quem é o responsável?	Projeção, WHERE, ORDER BY
C02	Básica	Quem mora nas Asas (Norte/Sul)?	LIKE
C03	Básica	Que mensalidades venceram desde 01/08 e seguem sem pagamento?	BETWEEN, IS NULL
C04	Básica	Quais turmas de 2026/2 são às segundas, quartas ou sextas?	IN
C05	Básica	Quem está com cadastro incompleto?	COALESCE, IS NULL
C06	Junção	Alunos ativos de cada turma, com nível e professor	Junção de 5 tabelas
C07	Junção	Turmas abertas por nível, incluindo níveis sem turma	LEFT JOIN
C08	Junção	Turmas de 2026/2 com menos da metade das vagas ocupadas	LEFT JOIN, GROUP BY, HAVING
C09	Agregação	Receita por mês e forma de pagamento	GROUP BY, SUM, COUNT
C10	Agregação	Turmas com frequência média abaixo de 85%	GROUP BY, HAVING
C11	Avançada	Quem foi aprovado em 2026/1 (média ≥ 6 e frequência ≥ 75%)?	Pergunta não trivial, tabelas derivadas
C12	Avançada	Quem teve média acima da média da própria turma?	Subconsulta correlacionada
C13	Avançada	Quem concluiu um nível e continuou no semestre seguinte?	EXISTS
C14	Avançada	Quem está inadimplente, quanto deve e há quantos dias?	Pergunta não trivial
C15	Avançada	Alguma regra atendida “por consulta” está sendo violada?	NOT EXISTS, UNION ALL

Teste em base limpa

> Os três scripts foram executados em sequência sobre um servidor MySQL 8.4.9 recém-inicializado, depois de `DROP DATABASE`, sem nenhum erro. As 15 consultas retornaram resultado, e a auditoria C15 retornou zero violações em todas as regras. Comandos:
>
> ```
> mysql -u root < sql/01_ddl.sql
> mysql -u root < sql/02_carga.sql
> mysql -u root --table < sql/03_consultas.sql
> ```

8. Declaração de uso de inteligência artificial

Em atendimento à seção 9 do enunciado, a equipe declara que usou o assistente de IA **Claude (Anthropic)**, via Claude Code, como apoio nas seguintes partes:

- **Modelagem:** discussão das alternativas de mapeamento da especialização, das chaves e da normalização, e redação inicial das justificativas das seções A4 e A5.
- **Scripts:** escrita inicial do 01_ddl.sql, do 03_consultas.sql e dos geradores em Python (carga de dados, diagrama e documentos).
- **Documentação:** rascunho do dicionário de dados e deste relatório.

Como o resultado foi verificado:

- Execução real dos três scripts em base limpa, no MySQL 8.4.9.
- Consultas de conferência sobre a carga. Elas encontraram, e levaram a corrigir, dois erros da primeira versão: uma coluna curta demais para o valor INTERMEDIARIO, e alunos reprovados por falta que apareciam matriculados no nível seguinte.
- Consulta de auditoria C15.
- Revisão, pela equipe, de cada decisão de modelagem, para que todos os integrantes consigam explicá-la na arguição.

Relatório da Etapa 2 — Operação, interface e desempenho

Projeto Final de Laboratório de Banco de Dados — UCB 2026/2 — Prof. Samuel Novais
Tema: Escola de Música · **SGBD:** MySQL 8 · **Aplicação:** Python 3 + Flask (web)

Integrantes: Gabriel Gomes Giani, Paulo Henrique Barreto da Rocha Dantas, Letícia Paolino, Artur Henrique Caetano e Lucas de Oliveira Faustino.

Este relatório cobre os itens B1 a B8 da Etapa 2. No arquivo `docs/relatorio-final.pdf` ele vem depois do relatório completo da Etapa 1 (A1 a A5), formando o relatório consolidado pedido no item B8. As alterações feitas no modelo desde a Etapa 1 estão na seção 9.

---

1. Como reproduzir tudo do zero

Passo	Comando	O que faz
1	`sql/01_ddl.sql`	Cria o banco e as 15 tabelas da Etapa 1
2	`sql/02_carga.sql`	Carga de dados fictícios
3	`sql/03_consultas.sql`	As 15 consultas da Etapa 1
4	`sql/04_seguranca.sql`	USUARIO_APP, visões e papéis (B5)
5	`sql/05_auditoria.sql`	Tabela AUDITORIA e gatilhos (B6)
6	`sql/06_otimizacao.sql`	EXPLAIN antes/depois e índices (B4)
7	`sql/07_transacoes.sql` (com `--force`)	COMMIT, ROLLBACK e roteiro de concorrência (B3)
8	`python ferramentas/criar_usuarios_banco.py`	Cria os usuários do MySQL, com as senhas do `.env`
9	`python app/app.py`	Sobe a aplicação em http://127.0.0.1:5000

O passo a passo detalhado, inclusive para o laboratório da faculdade, está no `README.md`. As saídas reais de cada script, geradas na execução que fundamenta este relatório, estão em `docs/resultados-*.txt`.

---

2. B1 — Aplicação com interface

A aplicação é web, feita em Flask, com páginas HTML simples. Toda a lógica de banco fica em `app/repositorio.py`.

Exigência	Onde está	Tela
CRUD completo de 4 entidades	ALUNO (PESSOA + ALUNO), TURMA, INSTRUMENTO e SALA	Alunos, Turmas, Instrumentos, Salas
2 consultas parametrizadas com junção e agregação	Desempenho por instrumento; Inadimplência no período	Consulta: desempenho / inadimplência
Relatório alimentado por visão	`vw_relatorio_financeiro` e `vw_ocupacao_turma`	Relatório
Operação que altera 2+ tabelas em uma transação	Matrícula: MATRICULA + HISTORICO_SITUACAO + MENSALIDADE	Matrícula
Erro de restrição tratado	`app/erros.py`	Qualquer tela

As duas consultas parametrizadas

1. **Desempenho por instrumento.** Filtros: instrumento e média mínima. A consulta junta AVALIACAO, MATRICULA, `vw_aluno_contato`, TURMA e NIVEL, agrupa por aluno e turma e filtra com `HAVING AVG(nota) >= ?`. O nome do aluno vem da visão, e não da tabela PESSOA, por isso a mesma tela funciona também para o perfil PROFESSOR, que não tem acesso a PESSOA.
2. **Inadimplência no período.** Filtros: vencimento de/até e total devido mínimo. A consulta junta MENSALIDADE, MATRICULA e PESSOA, filtra as mensalidades em aberto e já vencidas, agrupa por aluno e filtra com `HAVING SUM(valor) >= ?`.

Tratamento de erros. O usuário nunca vê a exceção do driver. `app/erros.py` lê o **nome da restrição** violada e devolve a mensagem da regra de negócio. Por isso as restrições foram nomeadas na Etapa 1. Exemplos testados na aplicação:

Ação	Erro do MySQL	Mensagem exibida
Cadastrar CPF repetido	1062 `uq_pessoa_cpf`	Já existe uma pessoa cadastrada com este CPF (RN01).
CPF com 3 dígitos	3819 `ck_pessoa_cpf`	O CPF deve ter exatamente 11 dígitos, só números (RN01).
Turma com 50 vagas	3819 `ck_turma_vagas`	A turma deve ter de 1 a 20 vagas (RN13).
Excluir aluno com matrícula	1451 `fk_matricula_aluno`	Não é possível excluir: o aluno ou a turma tem matrículas (RN16).
Matricular duas vezes	1062 `uq_matricula_aluno_turma`	Este aluno já está matriculado nesta turma (RN14).
Secretaria cadastra instrumento	1142 (sem permissão)	Seu perfil não tem permissão para esta operação (o banco recusou).

Duas regras que o MySQL não consegue expressar em CHECK são verificadas pela aplicação antes de gravar. A **RN03** exige que a data de nascimento não seja futura, e o MySQL não aceita `CURDATE()` dentro de CHECK. A **RN04** exige responsável para menor de idade.

---

3. B2 — Camada de acesso a dados

Consultas parametrizadas. Todo valor digitado pelo usuário entra no SQL como parâmetro (`%s`), e o driver PyMySQL o envia separado do comando. A aplicação não monta nenhum SQL concatenando texto do usuário. Nomes de tabela e coluna não podem ser parâmetro. Nos poucos pontos em que eles variam, no CRUD genérico, os nomes vêm de uma lista fixa definida no código (`ENTIDADES`), nunca da requisição.

Demonstração de tentativa de injeção de SQL (tela Alunos, campo de busca):

- Entrada digitada: `' OR 1=1 --`
- O que a aplicação executa: `... WHERE p.nome LIKE %s`, com o parâmetro `"' OR 1=1 -- %"`. O banco procura um nome que **começa com esse texto**.
- **Resultado obtido: 0 linhas.**
- Para comparar: se o SQL fosse montado por concatenação, o comando viraria `WHERE p.nome LIKE '' OR 1=1 -- %'`. O `OR 1=1` vale para todas as linhas e o `--` comenta o resto. Executado diretamente no MySQL, esse comando devolveu **os 47 alunos**.

Credenciais fora do código. As senhas do banco e a chave da sessão ficam no arquivo `.env`, que está no `.gitignore`. O repositório traz só o modelo, `.env.example`. Os usuários do MySQL também não são criados por script SQL, porque a senha ficaria versionada. Eles são criados por `ferramentas/criar_usuarios_banco.py`, que lê as senhas do `.env`.

---

4. B3 — Transações e controle de concorrência

4.1 A operação transacional

Matricular aluno (`repositorio.matricular()` na aplicação e `sql/07_transacoes.sql` no terminal):

```
START TRANSACTION
  SELECT vagas, id_nivel, data_fim FROM turma WHERE id_turma = ? FOR UPDATE   -- trava a turma
  SELECT COUNT(*) FROM matricula WHERE id_turma = ? AND situacao = 'ATIVA'    -- RN13 vagas
  (verifica pré-requisito concluído - RN28)
  INSERT INTO matricula ...
  INSERT INTO historico_situacao ...                                         -- RN17
  INSERT INTO mensalidade ...
COMMIT        (ou ROLLBACK em qualquer erro)
```

As três escritas precisam acontecer juntas. Matrícula sem mensalidade é aluno estudando sem cobrança. Mensalidade sem matrícula é cobrança de quem não estuda.

4.2 Falha provocada no meio da transação

O terceiro INSERT usa valor 0 e viola `ck_mensalidade_valor` (RN24). Resultado real (`docs/resultados-transacoes.txt`):

Momento	Matrículas do aluno 24 na turma 8
Dentro da transação, depois do erro	**1**
Depois do ROLLBACK	**0**
Históricos órfãos no banco	**0**

Ponto importante observado: no MySQL, um erro desfaz **só o comando que falhou**, não a transação inteira. A matrícula inserida antes continuava existindo dentro da sessão. Quem a desfaz é o `ROLLBACK` explícito, que na aplicação fica no bloco `except`. Sem ele, o próximo `COMMIT` gravaria uma matrícula sem mensalidade. Na aplicação, o mesmo teste (valor 0 na tela Matrícula) mostrou a mensagem de erro e não deixou nenhum registro.

4.3 Experimento com duas sessões simultâneas

Turma 7: 8 vagas, 7 ocupadas, sobra **uma**. Dois alunos tentam a última vaga ao mesmo tempo. O experimento foi executado com duas conexões reais em paralelo (`ferramentas/experimento_concorrencia.py`). Saída completa em `docs/resultados-concorrencia.txt`.

Nível de isolamento em vigor: REPEATABLE READ (padrão do InnoDB).

Caso	Comportamento observado	Resultado
Sem trava (só `SELECT COUNT`)	As duas sessões leram 7 ocupadas. Nenhuma esperou, as duas inseriram e fizeram COMMIT.	**9 matrículas numa turma de 8 vagas**
Com trava (`SELECT ... FOR UPDATE` na turma)	A sessão B **ficou bloqueada 1,48 s**, até o COMMIT da A. Depois viu 8 ocupadas e fez ROLLBACK.	**8 matrículas, correto**

Se a sessão A demorar mais que `innodb_lock_wait_timeout` (50 s), a B recebe o erro 1205, que a aplicação traduz como "Outra operação está usando este registro".

Justificativa do nível de isolamento. Mantivemos REPEATABLE READ, por três motivos:

- O problema do caso "sem trava" não é o nível de isolamento. É o padrão "ler, decidir e gravar" sem trava. READ COMMITTED teria o mesmo defeito.
- SERIALIZABLE evitaria o defeito sozinho, mas transforma toda leitura em leitura com trava compartilhada. As telas de consulta e o relatório passariam a bloquear e a ser bloqueados pelas matrículas.
- Com REPEATABLE READ e `FOR UPDATE` só na linha da turma, a trava fica exatamente onde existe conflito: duas matrículas na **mesma** turma. Matrículas em turmas diferentes continuam em paralelo.

Detalhe de implementação: o `FOR UPDATE` é a **primeira** leitura da transação. No REPEATABLE READ, a "foto" das leituras comuns é tirada na primeira leitura comum. Travando antes, a

contagem de vagas já enxerga o que a outra sessão gravou.

---

5. B4 — Otimização de consultas

Volume: com a carga da Etapa 1 (54 pessoas, 246 mensalidades), toda consulta roda abaixo de 1 ms e índice nenhum faz diferença mensurável. Por isso `06_otimizacao.sql` gera volume sintético (**50.054 pessoas** e **30.918 mensalidades**), mede, e no fim apaga os dados sintéticos. Os índices ficam. Os tempos abaixo são o `actual time` do `EXPLAIN ANALYZE` no MySQL 8.0. A saída completa está em `docs/resultados-otimizacao.txt`.

Consulta	Plano antes	Índice criado	Plano depois	Tempo antes	Tempo depois
1. Busca por início do nome	Table scan (50.054 linhas) + sort	`idx_pessoa_nome (nome)`	Covering index range scan (416 linhas)	12,3 ms	**0,18 ms**
2. Mensalidades vencidas e não pagas	Index lookup em `data_pagamento` (4.394 linhas) + filtro	`idx_mensalidade_aberto_venc (data_pagamento, data_vencimento)`	Index range scan (1.217 linhas)	6,8 ms	**1,4 ms**
3. Relatório financeiro por mês	Table scan + tabela temporária	`idx_mensalidade_comp_cobertura (competencia, data_pagamento, valor)`	Covering index scan + group aggregate	16,6 ms	**8,2 ms**

Leitura dos planos e escolha das colunas

Consulta 1. Antes, `type = ALL`: o banco leu as 50 mil linhas, testou o `LIKE` em cada uma e ordenou o resultado. `LIKE 'texto%'`, com o curinga só no final, é uma faixa ordenada da árvore B, então o índice em `nome` serve para filtrar **e** para o `ORDER BY`. Como o InnoDB guarda a chave primária em todo índice secundário, `(nome)` já contém `id_pessoa`, e a consulta é respondida só pelo índice ("covering"). Observação: `LIKE '%texto'` não aproveitaria índice nenhum.

Consulta 2. Já existia um índice só em `data_pagamento`, da Etapa 1. Ele achava as 4.394 mensalidades em aberto, mas o banco ainda lia cada uma para testar o vencimento, e 72% eram descartadas. No índice composto, a **ordem importa**: `data_pagamento` vem primeiro porque o teste é de **igualdade** (IS NULL), e `data_vencimento` depois porque é **faixa** (<). Depois de uma coluna de faixa, as colunas seguintes do índice não filtram mais. O índice antigo virou prefixo do novo e foi removido, porque seria só custo de escrita.

Consulta 3 — resultado honesto. É o ganho mais fraco e o menos confiável. A consulta continua lendo **todas** as 30.918 linhas, porque é um relatório do histórico inteiro, e nenhum índice evita isso. O ganho veio de dois detalhes: ler um índice mais estreito que a tabela e agrupar na ordem do índice, sem tabela temporária. Em duas execuções completas do script, o ganho foi de **21%** (17,7 → 14,0 ms) e de **51%** (16,6 → 8,2 ms, a execução salva). Medições de poucos milissegundos variam com cache e carga da máquina. Por isso a conclusão segura é "melhora moderada", não um número exato. Nas consultas 1 e 2 a melhora se repetiu nas duas execuções, porque o número de linhas lidas caiu de verdade (de 50.054 para 416, e de 4.394 para 1.217). Para reduzir de verdade seria preciso ler menos linhas, por exemplo com uma tabela de resumo mensal mantida por gatilho. Nesta escala isso não se justifica, porque 14 ms com 30 mil linhas equivalem a muitos anos de operação da escola.

---

6. B5 — Segurança e controle de acesso

6.1 Perfis e privilégios (`sql/04_seguranca.sql`)

Cada perfil da aplicação conecta ao MySQL com **um usuário de banco diferente**. Quem garante as permissões é o próprio banco, não o menu. Mesmo que alguém burle a tela, o MySQL recusa.

Papel MySQL	Usuário	Pode	Não pode
r_login	app_login	Ler USUARIO_APP e atualizar ultimo_login	Qualquer outra tabela
r_admin	app_admin	SELECT/INSERT/UPDATE/DELETE em tudo	Criar/alterar estrutura (DDL) e usuários
r_secretaria	app_secretaria	Cadastro de alunos e turmas, matrícula, lançar e dar baixa em mensalidade	Instrumentos/salas (só leitura), **apagar mensalidade**, ler AUDITORIA
r_professor	app_professor	Ler turmas; lançar aulas, frequência e notas	Ler PESSOA (CPF, endereço), MENSALIDADE, AUDITORIA

O **REVOKE** demonstrado: a secretaria recebeu DELETE em MENSALIDADE junto com os outros privilégios, e o script o retira com REVOKE DELETE ON mensalidade FROM r_secretaria (RN33: só o administrador exclui mensalidade). Testes com login real:

```
app_professor> SELECT cpf FROM pessoa;              -> ERROR 1142 SELECT command denied
app_professor> SELECT * FROM vw_aluno_contato;      -> funciona (nome, e-mail, telefone, sem CPF)
app_secretaria> DELETE FROM mensalidade WHERE ...;  -> ERROR 1142 DELETE command denied
app_secretaria> SELECT * FROM auditoria;            -> ERROR 1142 SELECT command denied
```

O **mesmo login sob dois perfis** aparece na demonstração: a tela "Consulta: desempenho" funciona para SECRETARIA e PROFESSOR, mas a tela Alunos, com CPF, só funciona para a secretaria. Para o professor o banco recusa e a aplicação mostra a mensagem tratada.

6.2 Visão de restrição

vw_aluno_contato restringe **colunas**: expõe nome, e-mail, telefone, se é menor de idade e o responsável. Esconde CPF, endereço e data de nascimento completa. O professor recebe SELECT na visão e não na tabela PESSOA.

6.3 Senhas com hash e sal

Tabela USUARIO_APP: a senha nunca é gravada. Grava-se o **PBKDF2-HMAC-SHA256 com 200.000 iterações** sobre a senha, com um **sal aleatório de 16 bytes por usuário**.

- O sal faz duas senhas iguais gerarem hashes diferentes e inutiliza tabelas pré-calculadas.
- As 200 mil iterações deixam cada tentativa de força bruta lenta.
- A comparação usa hmac.compare_digest, em tempo constante.
- Login inexistente e senha errada recebem a **mesma** mensagem, para não revelar quais logins existem.

6.4 Dados pessoais no projeto

Dado	Onde	Por que é sensível	Cuidado adotado
CPF	PESSOA	Identifica a pessoa	Só ADMIN e SECRETARIA leem. Professor vê pela visão, sem CPF. Não é chave primária (Etapa 1), então não se espalha pelas tabelas filhas.
Nome, nascimento, e-mail, endereço, telefone	PESSOA, TELEFONE	Dados pessoais (LGPD)	Mesmo controle do CPF. Endereço e nascimento ficam fora da visão do professor.
Responsável e dados de menor de idade	ALUNO	Dados de criança e adolescente pedem cuidado reforçado	Acesso restrito como acima. O professor vê só o indicador "menor de idade" e o nome do responsável, necessário para contato.

Dado	Onde	Por que é sensível	Cuidado adotado
Situação financeira	MENSALIDADE	Expõe a vida financeira do aluno	Professor não tem acesso. Alterações auditadas. Secretaria não apaga.
Senhas	USUARIO_APP	Credencial	Só hash + sal.
Credenciais do banco	`.env`	Acesso total aos dados	Fora do repositório.

Toda a carga é **fictícia**, gerada por `ferramentas/gerar_carga.py`. Nenhum dado real de terceiros foi usado.

---

7. B6 — Gatilhos e auditoria

`sql/05_auditoria.sql` cria a tabela AUDITORIA e dois gatilhos, AFTER UPDATE. Cada registro guarda **autor** (login da aplicação e usuário do MySQL), **data e hora**, e os **valores anterior e posterior** em JSON.

Gatilho	Tabela	Por que auditar (pelo domínio)
`trg_mensalidade_au`	MENSALIDADE	É dinheiro. Uma baixa lançada por engano ou um valor alterado vira prejuízo para a escola ou cobrança indevida ao aluno. (RN34)
`trg_matricula_au`	MATRICULA	Trancar ou cancelar libera vaga, interrompe cobrança e muda o histórico acadêmico. HISTORICO_SITUACAO diz o que mudou; a auditoria diz quem mudou. (RN35)

Por que não auditamos AULA e FREQUENCIA: são as tabelas de maior movimento e as alterações nelas não têm impacto financeiro. Auditar tudo gera volume sem uso.

Como o autor é registrado. A aplicação executa `SET @usuario_app = '<login>'` ao conectar, e o gatilho grava esse valor e o `USER()` do MySQL. Os gatilhos executam com os privilégios de quem os criou, então secretaria e professor não precisam de permissão na AUDITORIA e não podem lê-la nem apagá-la. Só o ADMIN vê a tela Auditoria.

Exemplo real, uma baixa de pagamento pela tela Mensalidades com o login `secretaria`:

```
tabela: mensalidade   chave: 104   usuario_app: secretaria   usuario_banco: app_secretaria@localhost
antes:  {"valor": 280.00, "data_pagamento": null,         "data_vencimento": "2026-02-10", "forma_pagamento": null}
depois: {"valor": 280.00, "data_pagamento": "2026-10-26", "data_vencimento": "2026-02-10", "forma_pagamento": "PIX"}
```

---

8. B7 — Projeto físico e armazenamento

8.1 Revisão dos tipos de dados

Coluna	Tipo atual	Avaliação
`valor` (mensalidade), `valor_hora`	DECIMAL(8,2) e DECIMAL(7,2)	**Mantém.** Dinheiro nunca em FLOAT, que arredonda (0,1 + 0,2 ≠ 0,3).
`cpf`, `cep`, `uf`	CHAR(11), CHAR(8), CHAR(2)	**Mantém.** Tamanho fixo, e CHAR evita o byte de comprimento.
`situacao`, `tipo`, `familia`, `status`	VARCHAR + CHECK	**Mantém.** ENUM ocuparia 1 byte, mas incluir um valor novo exige ALTER TABLE e reordenar o ENUM muda o significado dos dados. Com CHECK, a mudança é só na restrição.
`id_sala` (TINYINT), `id_instrumento`/`id_nivel` (SMALLINT)		**Mantém.** Uma escola não terá mais que 255 salas nem 65 mil níveis. Chave menor deixa todo índice que a referencia menor.

Coluna	Tipo atual	Avaliação
`telefone.numero`	VARCHAR(15)	**Mudaria para CHAR(11) ou VARCHAR(11).** O CHECK já limita a 10–11 dígitos, então 15 nunca é usado.
`aula.numero_aula`	TINYINT UNSIGNED	**Mantém, com atenção.** Limite de 255 aulas por turma. Cursos longos (anuais com 3 aulas por semana) chegariam perto.
`auditoria.id_auditoria`	BIGINT	**Novo nesta etapa.** Tabela que só cresce e nunca é apagada.
`historico_situacao.data_alteracao`	DATETIME	**Mantém.** DATETIME não sofre com fuso horário como TIMESTAMP e vai além de 2038.

Índices revisados nesta etapa: entraram `idx_pessoa_nome`, `idx_mensalidade_aberto_venc` e `idx_mensalidade_comp_cobertura` (B4). O `idx_mensalidade_pagamento`, da Etapa 1, saiu por ser redundante.

8.2 Estimativa de crescimento (2 anos)

Base: a carga atual tem cerca de 37 matrículas por semestre (74 em dois semestres), com cerca de 13 aulas registradas por turma. Hipótese: crescimento de **20% de matrículas por semestre**, de 37 para 44, 53, 64 e 77, um total de **cerca de 240 matrículas novas em 2 anos**. Por matrícula e semestre: 18 aulas com frequência, 5 mensalidades, 2 avaliações, 2 eventos de histórico e cerca de 5 registros de auditoria.

Tabela	Linhas hoje	Linhas novas em 2 anos	Bytes/linha (dado + índices)	Tamanho em 2 anos
FREQUENCIA (maior movimento)	901	~4.300	~60	< 1 MB
MENSALIDADE	246	~1.200	~120 (3 índices)	< 1 MB
AUDITORIA	0	~1.200	~400 (JSON)	< 1 MB
AVALIACAO, HISTORICO_SITUACAO	127 / 111	~500 cada	~100	< 1 MB

Conclusão: em 2 anos o banco inteiro fica na casa de **poucos megabytes**. O tamanho que o MySQL mostra hoje, de 16 a 48 KB por tabela, é quase só a página mínima de 16 KB do InnoDB, não dado.

8.3 Particionamento e arquivamento

Nenhuma tabela justifica particionamento. Particionar compensa quando uma tabela tem dezenas de milhões de linhas, ou quando se apaga um período inteiro de uma vez. Aqui a maior tabela teria cerca de 5 mil linhas em 2 anos. Além disso, no MySQL a chave de partição precisa fazer parte de toda chave primária e única, o que obrigaria a mudar as PKs, e as tabelas com chave estrangeira não podem ser particionadas.

Proposta para o futuro, só se a escola virar uma rede: **arquivar FREQUENCIA de semestres encerrados** em uma tabela `frequencia_arquivo`, uma vez por ano, já que o dado é imutável depois que o semestre fecha.

8.4 Plano de backup

Item	Definição
O que copiar	O banco `escola_musica` inteiro, com gatilhos e visões: `mysqldump --single-transaction --routines --triggers --events escola_musica`. Também os papéis e usuários do MySQL, que ficam fora do banco: reaplicar `04_seguranca.sql` e `criar_usuarios_banco.py`. O `.env` vai para um cofre de senhas, **não** junto do backup.

Item	Definição
Frequência	Backup **completo diário** às 23h, depois do expediente (pela RN11 a última aula começa às 21h). **Log binário** (binlog) ligado, para recuperar até o minuto anterior a uma falha (point-in-time recovery).
Retenção	Avô-pai-filho: **7 diários**, **4 semanais** (domingo) e **12 mensais** (dia 1). Os mensais cobrem o ano fiscal de mensalidades.
Onde guardar	Uma cópia local e uma em outro lugar (nuvem), criptografada, porque o backup contém dados pessoais.
Teste de restauração	Todo mês: restaurar o backup do dia em um MySQL separado, rodar `03_consultas.sql` e conferir se a consulta C15 (auditoria de regras) retorna 0 em todas as linhas e se a contagem de linhas bate com a produção. O backup só conta como feito se a restauração funcionar.

`--single-transaction` faz o dump ler uma foto consistente sem travar as tabelas. Funciona porque todas são InnoDB, e as matrículas continuam funcionando durante o backup.

---

9. Alterações no modelo desde a Etapa 1

Correções da devolutiva da Etapa 1: não há correções de devolutiva registradas nesta versão. As mudanças feitas no modelo desde a Etapa 1 estão listadas abaixo.

O que mudou no modelo nesta etapa:

Mudança	Motivo
Nova entidade **USUARIO_APP** (login, senha_hash, sal, perfil, id_pessoa)	Login da aplicação com hash e sal (B5)
Nova entidade **AUDITORIA**	Registro de alterações (B6)
Visões `vw_aluno_contato`, `vw_ocupacao_turma`, `vw_relatorio_financeiro`	Restrição de acesso e relatório (B1, B5)
Índices novos e remoção de `idx_mensalidade_pagamento`	Otimização (B4)
`SET NAMES utf8mb4` no início dos scripts	Evita acentos corrompidos ao rodar pelo terminal com codificação diferente de UTF-8
Novas regras **RN31 a RN35**	Ver abaixo

Novas regras de negócio

Regra	Texto	Atendida por
RN31	O login de usuário da aplicação é único.	Restrição `uq_usuario_app_login`
RN32	Todo usuário da aplicação tem exatamente um perfil: ADMIN, SECRETARIA ou PROFESSOR.	Restrição `ck_usuario_app_perfil`
RN33	Somente o administrador exclui mensalidade.	`REVOKE DELETE ... FROM r_secretaria`
RN34	Toda alteração de mensalidade fica registrada com autor, data e valores.	Gatilho `trg_mensalidade_au`
RN35	Toda mudança de situação da matrícula fica registrada com autor, data e valores.	Gatilho `trg_matricula_au`

Dicionário de dados das novas entidades

USUARIO_APP

Atributo	Descrição	Domínio	Obrig.	Observação
id_usuario	Identificador interno	Inteiro sequencial	Sim	Chave primária substituta
login	Nome de acesso	Texto até 30	Sim	RN31 — único
senha_hash	PBKDF2-SHA256 da senha	64 caracteres hexadecimais	Sim	Nunca a senha em texto
sal	Sal aleatório do usuário	32 caracteres hexadecimais	Sim	16 bytes
perfil	Perfil de acesso	ADMIN, SECRETARIA, PROFESSOR	Sim	RN32
id_pessoa	Professor ligado ao login	Inteiro (FK PESSOA)	Não	ON DELETE SET NULL
ativo	Login liberado	Booleano	Sim	Padrão verdadeiro
ultimo_login	Último acesso	Data e hora	Não	

AUDITORIA

Atributo	Descrição	Domínio	Obrig.	Observação
id_auditoria	Identificador	Inteiro sequencial (BIGINT)	Sim	Chave primária
tabela	Tabela alterada	Texto até 30	Sim	
operacao	Tipo de alteração	INSERT, UPDATE, DELETE	Sim	
chave	Chave do registro alterado	Texto até 40	Sim	
valores_antes / valores_depois	Estado anterior e posterior	JSON	Não	RN34, RN35
usuario_app	Login da aplicação	Texto até 30	Não	Nulo se veio do terminal
usuario_banco	Usuário do MySQL	Texto até 100	Sim	USER()
data_hora	Momento da alteração	Data e hora com milissegundos	Sim	Padrão: agora

---

10. Roteiro da demonstração (15 min, item 6.2)

Antes da aula: banco carregado (passos 1 a 8 da seção 1), aplicação rodando e dois terminais MySQL abertos.

Momento	Tempo	O que fazer
Contexto	2 min	Domínio, entidades principais e o que mudou desde a Etapa 1 (seção 9)
CRUD e consultas	4 min	Login `secretaria` → cadastrar aluno → Consulta: inadimplência → Relatório
Transação	3 min	Matrícula com valor 320 (sucesso). Matrícula com valor 0 (falha): mostrar que nada ficou gravado
Restrições em ação	2 min	CPF repetido; excluir aluno com matrícula; turma com 50 vagas
Desempenho e acesso	3 min	`06_otimizacao.sql`: EXPLAIN antes e depois da consulta 1. Login `prof.julia` → tela Alunos recusada pelo banco; desempenho funciona. Nos dois terminais: caso 3b de `07_transacoes.sql`
Arguição	1 min	—

11. Declaração de uso de inteligência artificial (item 9 do enunciado)

A equipe usou o assistente de IA **Claude (Anthropic)**, pelo Claude Code, nesta etapa:

- **Onde foi usado:** redação e revisão dos scripts `04` a `07`, da aplicação Flask (`app/`), das ferramentas `criar_usuarios_banco.py` e `experimento_concorrencia.py` e deste relatório.
- **Como o resultado foi verificado:**
  - Todos os scripts foram executados do início ao fim em um MySQL 8.0 com o banco recriado do zero (`01` → `07`).
  - As saídas reais foram salvas em `docs/resultados-*.txt`. Os números deste relatório (tempos, contagens, resultados de concorrência) foram copiados dessas execuções, não estimados, exceto a estimativa de crescimento da seção 8.2, que é declaradamente uma hipótese.
  - A aplicação foi testada nos três perfis, com casos de sucesso e de erro (seção 2).
- **Responsabilidade:** a equipe revisou o material e é responsável por explicar qualquer decisão na arguição.